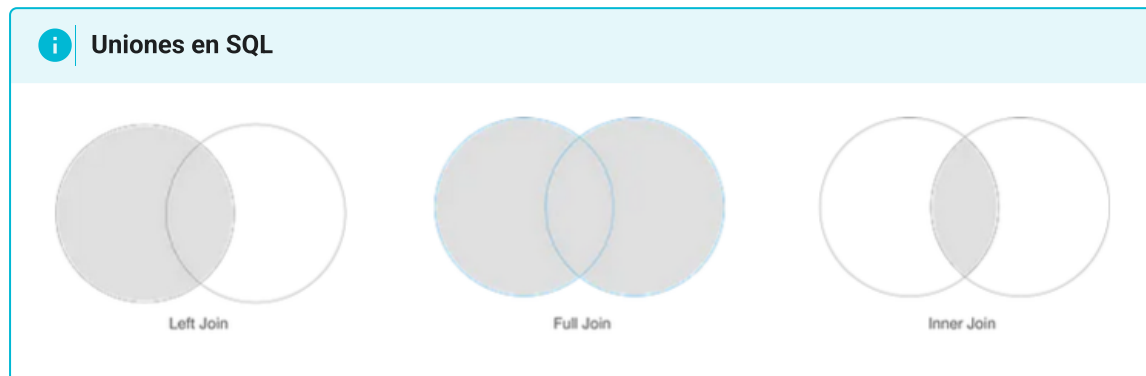


KSQL Joins



ksqlDB soporta tres tipos de *join* (más info en [Join Capabilities](#)):

- **Stream-Stream Joins:** ksqlDB soporta INNER, LEFT OUTER (también llamado LEFT JOIN), RIGHT OUTER, y FULL OUTER (también llamado OUTER JOIN) **joins entre streams**. Estos joins se definen sobre ventanas de datos de acuerdo con la naturaleza continua de los streams. Cada ventana define el período de tiempo durante el cual los mensajes son considerados válidos para el join. Con cada mensaje se asocia un período de gracia que determina el lapso de tiempo admitido fuera de la ventana de tiempo. Para más información al respecto ver [Time and Windows](#)
- **Stream-Table Joins:** ksqlDB soporta INNER y LEFT joins entre un stream y una table. OUTER joins no son soportados. Este tipo de join actúa como un buscador en la tabla lo cual permite enriquecer el stream con los datos que contiene. Las uniones flujo-tabla siempre son sin ventana
- **Table-Table Joins:** ksqlDB admite uniones (1:1) y (1:N) entre tablas, pero no uniones de muchos a muchos (N:M). Para las uniones de clave externa, se puede utilizar cualquier columna de la tabla izquierda en la condición de unión con la clave primaria de la tabla derecha. Las uniones INNER, LEFT OUTER y FULL OUTER son compatibles con las uniones de clave primaria. Mientras que las uniones N-way son generalmente soportadas, las uniones FULL OUTER N-way requieren múltiples consultas separadas, cada una realizando una única unión FULL OUTER

```
-- Ejemplo de union stream-table
CREATE STREAM pageviews_enriched AS
SELECT users.userid AS userid, pageid, regionid, gender
FROM pageviews
LEFT JOIN users ON pageviews.userid = users.userid
EMIT CHANGES;
```

```
-- Ejemplo de unión entre tres streams
```

```
CREATE STREAM shipped_orders AS
SELECT
  o.id AS orderId,
  o.itemid AS itemId,
  s.id AS shipmentId,
  p.id AS paymentId
FROM orders o
  INNER JOIN payments p WITHIN 1 HOURS ON p.id = o.id
  INNER JOIN shipments s WITHIN 2 HOURS ON s.id = o.id;
```

Requisitos del Join (más detalle en [Partition Data to Enable Joins](#))

Para unir streams estos deberán estar coparticionados (ver información detallada en [Co-partitioning Requirements](#)) lo que significa que los registros de entrada de ambos lados de la unión tienen el mismo esquema de particionados los mismos ajustes de configuración para las particiones. Las uniones entre tablas no tienen ningún requisito de copartición. Esto es debido a que para unir dos fuentes de datos, flujos o tablas, ksqlDB necesita comparar sus registros basándose en la columna de unión. Para garantizar que los registros con la misma columna de unión estén ubicados en la misma tarea del flujo, la columna de unión debe coincidir con la columna por la que se particionan las fuentes

Las tablas siempre se particionan por su clave primaria y ksqlDB no permite la repartición de tablas lo que significa que sólo se puede utilizar la clave primaria de una tabla como columna de unión. Los streams no tienen claves primarias pero tienen una columna KEY opcional. Una columna KEY, cuando está presente, define la columna de partición

Los flujos permiten uniones en expresiones distintas de su columna clave. Cuando los criterios de unión difieren de la columna KEY, ksqlDB reparte internamente el flujo, lo que implícitamente define la clave y el particionado correctos

El siguiente ejemplo muestra una tabla *users* unida con un stream *clicks* por la columna *userId*. La tabla tiene una clave primaria *id* del mismo tipo SQL. El stream no tiene una clave definida por lo que ksqlDB lo reparticiona automáticamente en la columna de unión (*userId*) para asignar la clave antes de realizar la unión

```
-- clicks stream, with no or unknown key.
-- the schema of stream clicks is: USERID BIGINT | URL STRING
CREATE STREAM clicks (
  userId BIGINT,
  url STRING
) WITH (
  kafka_topic='clickstream',
  value_format='json',
  partitions=1
);

-- users table, with userId primary key.
```

```
-- the schema of table users is: USERID BIGINT PRIMARY KEY | FULLNAME STRING
CREATE TABLE users (
  id BIGINT PRIMARY KEY,
  fullName STRING
) WITH (
  kafka_topic='users',
  value_format='json',
  partitions=1
);

-- join of users table with clicks stream, joining on the table's primary key
and the stream's userId column:
-- join will automatically repartition clicks stream:
SELECT
  c.userId,
  c.url,
  u.fullName
FROM clicks c
  JOIN users u ON c.userId = u.id
EMIT CHANGES;
```